

MANUAL TÉCNICO

Sistema de Gestión de Proyectos — Constructora GG

Evidencia de producto GA10-220501097-AA10-EV01

Elabora documentos técnicos y de usuario del software

Servicio Nacional de Aprendizaje – SENA

Centro de Comercio y Servicios · Ficha **3134547**

Programa: Análisis y Desarrollo de Software (ADSO)

Aprendiz:

Cristian Gutierrez

Jesús Romero

2026

Introducción

Este manual técnico documenta el sistema Constructora GG, una plataforma de gestión de proyectos de construcción desarrollada como proyecto formativo SENA ADSO. El documento reúne los prerequisites de instalación, los frameworks y estándares utilizados, los diagramas de análisis y diseño (casos de uso, entidad-relación y componentes), el diccionario de datos de la base de datos y los scripts necesarios para desplegar el sistema.

Toda la información aquí presentada fue extraída directamente del código fuente del sistema (backend, frontend y base de datos), garantizando que el manual refleje la implementación real y no una versión conceptual o preliminar del proyecto.

Prerrequisitos de instalación del sistema

Para desplegar el sistema en un equipo local, solo se necesita tener instalado:

- Docker — versión 24.x o superior (<https://docs.docker.com/get-docker/>)
- Docker Compose — v2, incluido en Docker Desktop

No se requiere instalar Python, Node.js, ni ninguna otra dependencia directamente en el equipo anfitrión: todo el entorno de ejecución (backend, frontend y base de datos) se encuentra empaquetado dentro de los contenedores Docker.

Verificación de requisitos

```
docker --version
docker compose version
```

Puertos utilizados

- 8000 → Backend (API REST — FastAPI / Uvicorn)
- 8080 → Frontend (interfaz web servida por Nginx)

Antes de iniciar el sistema se debe verificar que estos puertos estén libres en el equipo.

Credenciales de prueba

Usuario	Rol	Contraseña
admin	Administrador	admin123
david.lider	Líder de Proyecto	lider123
pedro.operario	Operario	operario123

Frameworks y estándares con los que se construyó el sistema

Backend

- FastAPI (Python 3.11) — framework web para la construcción de la API REST
- SQLAlchemy — ORM para el acceso y modelado de la base de datos
- Pydantic / pydantic-settings — validación de datos y gestión de configuración
- Uvicorn — servidor ASGI, con recarga automática en desarrollo

Base de datos

- SQLite — base de datos embebida, no requiere motor externo (archivo database/constructora_gg.db)

Seguridad

- JWT (python-jose) — autenticación basada en tokens, con esquema OAuth2PasswordBearer
- Bcrypt (passlib) — cifrado de contraseñas
- Control de acceso por roles (Admin / Líder de Proyecto / Operario) mediante dependencias de FastAPI

Frontend

- HTML5, CSS3 (variables y grid) y JavaScript ES6+ vanilla — sin frameworks como React o Vue
- Arquitectura modular propia: componentes, módulos y servicios independientes

Infraestructura y despliegue

- Docker y Docker Compose — contenedores independientes para backend y frontend
- Nginx — servidor del frontend estático
- python-dotenv — gestión de variables de entorno (.env)

Pruebas

- Pytest — pruebas automatizadas del backend (autenticación, usuarios, proyectos)

Estándares de ingeniería de software aplicados

- Documento de Especificación de Requisitos de Software (SRS)
- Diagramas UML: casos de uso, clases, componentes y despliegue
- Diagrama Entidad-Relación y Diagrama Relacional de la base de datos
- Colección Postman para documentación y pruebas de la API

Diagrama de casos de uso del sistema

El sistema cuenta con tres roles: Administrador, Líder de Proyecto y Operario. A continuación se presenta el diagrama de casos de uso, construido a partir de los endpoints reales de la API y sus restricciones de acceso por rol.

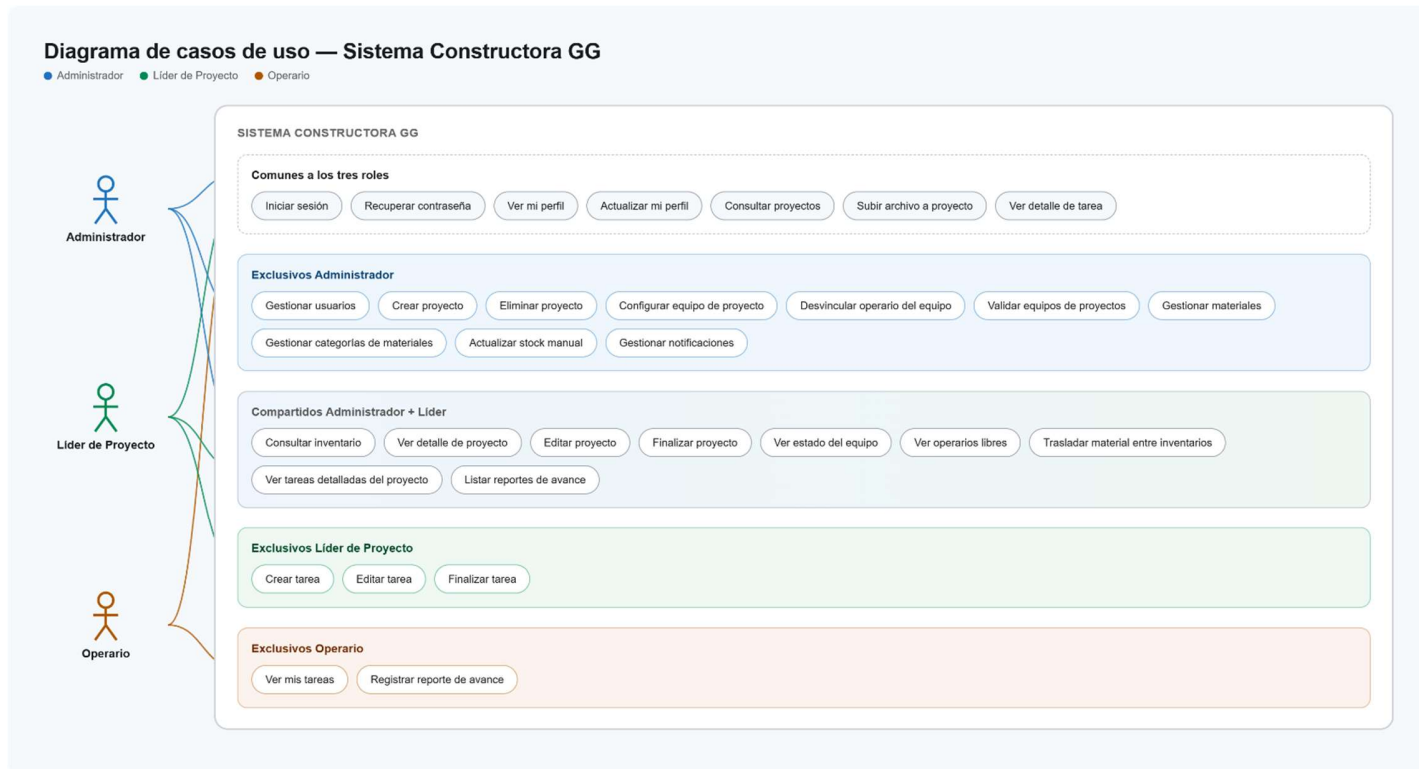


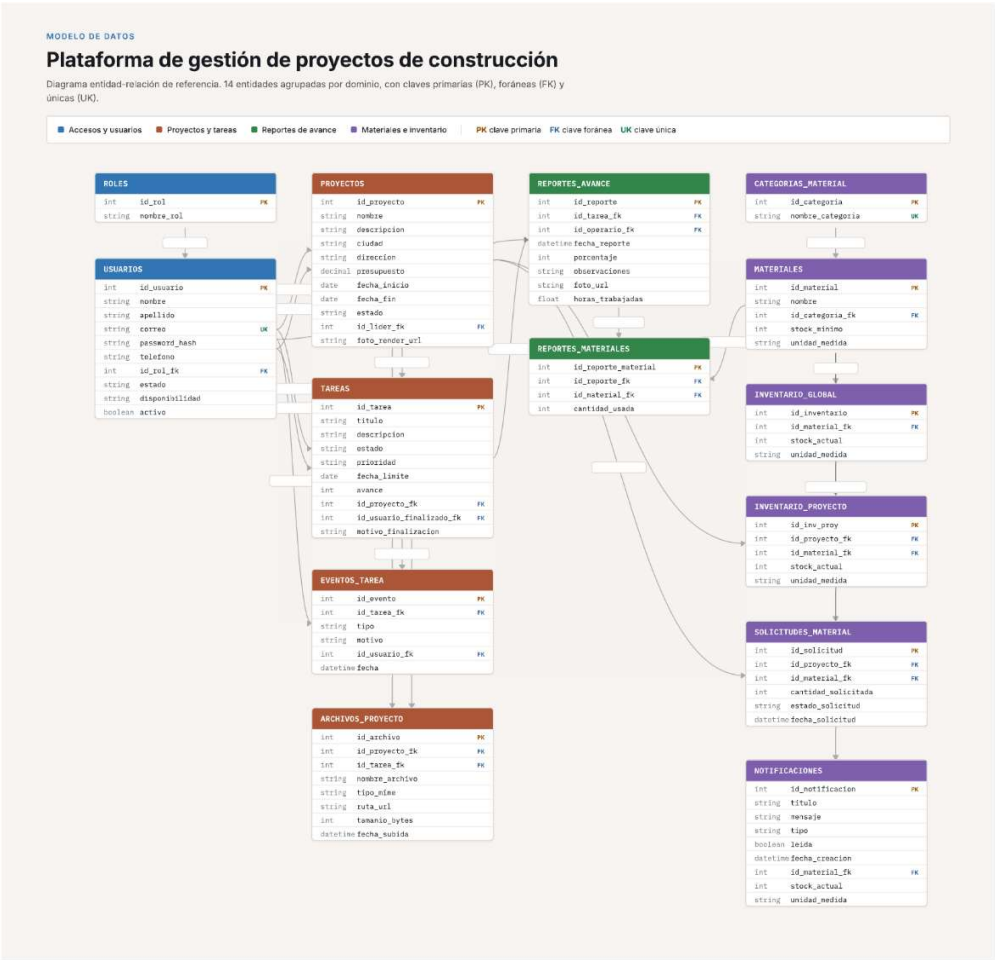
Diagrama de casos de uso — Sistema Constructora GG

Resumen por rol

- Comunes a los tres roles: iniciar sesión, recuperar contraseña, gestionar perfil propio, subir archivos a proyecto, consultar proyectos e inventario, ver detalle de tarea.
- Administrador: gestión de usuarios, proyectos, equipos de trabajo, materiales, inventario y notificaciones.
- Líder de Proyecto: edición de proyectos, gestión de tareas, traslado de materiales entre inventarios, consulta de reportes de avance.
- Operario: consulta de tareas asignadas y registro de reportes de avance.

5. Modelo entidad-relación de la base de datos

El modelo entidad-relación fue construido a partir de los modelos de SQLAlchemy definidos en el backend (backend/app/models.py), reflejando la estructura real de la base de datos SQLite del



sistema.

Modelo entidad-relación — Base de datos Constructora GG

6. Diccionario de datos

A continuación se detalla cada tabla de la base de datos con sus campos, tipos de dato, restricciones y descripción.

roles — Catálogo de roles del sistema

Campo	Tipo de dato	Restricciones	Descripción
id_rol	INTEGER	PK, autoincremental	Identificador único del rol
nombre_rol	VARCHAR(50)	UNIQUE, NOT NULL	Nombre del rol: Admin, Lider u Operario

usuarios — Personas que acceden al sistema

Campo	Tipo de dato	Restricciones	Descripción
id_usuario	INTEGER	PK, autoincremental	Identificador único del usuario
nombre	VARCHAR(100)	NOT NULL	Nombre del usuario
apellido	VARCHAR(100)	NOT NULL	Apellido del usuario
correo	VARCHAR(100)	UNIQUE, NOT NULL, INDEX	Correo institucional, usado para iniciar sesión
password_hash	VARCHAR(200)	NOT NULL	Contraseña cifrada con Bcrypt
telefono	VARCHAR(20)	NULL	Teléfono de contacto
id_rol_fk	INTEGER	FK → roles.id_rol	Rol asignado al usuario
estado	VARCHAR(50)	DEFAULT 'activo'	activo inactivo vacaciones licencia
disponibilidad	VARCHAR(50)	DEFAULT 'disponible'	disponible ocupado
activo	BOOLEAN	DEFAULT TRUE	Indica si la cuenta está habilitada

proyectos — Proyectos de construcción gestionados

Campo	Tipo de dato	Restricciones	Descripción
id_proyecto	INTEGER	PK, autoincremental	Identificador único del proyecto
nombre	VARCHAR(255)	NOT NULL	Nombre del proyecto
descripcion	TEXT	NULL	Descripción del proyecto
ciudad	VARCHAR(100)	NULL	Ciudad donde se ejecuta
direccion	VARCHAR(255)	NULL	Dirección del frente de obra
presupuesto	NUMERIC(15,2)	NULL	Presupuesto asignado
fecha_inicio	DATE	DEFAULT fecha actual	Fecha de inicio del proyecto
fecha_fin	DATE	NULL	Fecha de finalización
estado	VARCHAR(50)	DEFAULT 'activo'	activo pausado finalizado
id_lider_fk	INTEGER	FK → usuarios.id_usuario, NOT NULL	Líder de proyecto responsable
foto_render_url	VARCHAR(500)	NULL	URL de imagen/render del proyecto

proyectos_usuarios — Tabla intermedia N:M — equipo de operarios por proyecto

Campo	Tipo de dato	Restricciones	Descripción
id_proyecto	INTEGER	PK, FK → proyectos.id_proyecto	Proyecto asociado
id_usuario	INTEGER	PK, FK → usuarios.id_usuario	Operario asociado al equipo

tareas — Tareas asignadas dentro de un proyecto

Campo	Tipo de dato	Restricciones	Descripción
id_tarea	INTEGER	PK, autoincremental	Identificador único de la tarea
titulo	VARCHAR(255)	NOT NULL	Título de la tarea
descripcion	TEXT	NULL	Descripción detallada
estado	VARCHAR(50)	DEFAULT 'pendiente'	pendiente en_progreso finalizada
prioridad	VARCHAR(50)	DEFAULT 'media'	baja media alta
fecha_limite	DATE	NULL	Fecha límite de entrega
avance	INTEGER	DEFAULT 0	Porcentaje de avance acumulado
id_proyecto_fk	INTEGER	FK → proyectos.id_proyecto	Proyecto al que pertenece
id_usuario_finalizado_fk	INTEGER	FK → usuarios.id_usuario, NULL	Usuario que finalizó la tarea
motivo_finalizacion	VARCHAR(100)	NULL	progreso_completado finalizado_por_lider

tareas_operarios — Tabla intermedia N:M — operarios asignados a cada tarea

Campo	Tipo de dato	Restricciones	Descripción
id_tarea	INTEGER	PK, FK → tareas.id_tarea	Tarea asociada
id_usuario	INTEGER	PK, FK → usuarios.id_usuario	Operario asignado

eventos_tarea — Historial de eventos de finalización/reactivación de tareas

Campo	Tipo de dato	Restricciones	Descripción
id_evento	INTEGER	PK, autoincremental	Identificador único del evento
id_tarea_fk	INTEGER	FK → tareas.id_tarea, NOT NULL	Tarea relacionada
tipo	VARCHAR(50)	NOT NULL	finalizada reactivada
motivo	VARCHAR(100)	NULL	progreso_completado finalizado_por_lider
id_usuario_fk	INTEGER	FK → usuarios.id_usuario, NOT NULL	Usuario que generó el evento
fecha	DATETIME	DEFAULT now()	Fecha y hora del evento

reportes_avance — Reportes de avance registrados por los operarios

Campo	Tipo de dato	Restricciones	Descripción
id_reporte	INTEGER	PK, autoincremental	Identificador único del reporte
id_tarea_fk	INTEGER	FK → tareas.id_tarea	Tarea reportada
id_operario_fk	INTEGER	FK → usuarios.id_usuario	Operario que reporta
fecha_reporte	DATETIME	DEFAULT now()	Fecha y hora del reporte
porcentaje	INTEGER	NOT NULL	Porcentaje de avance reportado
observaciones	TEXT	NULL	Observaciones del operario
foto_url	VARCHAR(500)	NULL	Evidencia fotográfica del avance
horas_trabajadas	FLOAT	DEFAULT 0.0	Horas invertidas en la jornada

reportes_materiales — *Materiales consumidos en cada reporte de avance*

Campo	Tipo de dato	Restricciones	Descripción
id_reporte_material	INTEGER	PK, autoincremental	Identificador único
id_reporte_fk	INTEGER	FK → reportes_avance.id_reporte	Reporte asociado
id_material_fk	INTEGER	FK → materiales.id_material	Material utilizado
cantidad_usada	INTEGER	NOT NULL	Cantidad de material consumida

categorias_material — *Catálogo de categorías de materiales*

Campo	Tipo de dato	Restricciones	Descripción
id_categoria	INTEGER	PK, autoincremental	Identificador único
nombre_categoria	VARCHAR(100)	UNIQUE, NOT NULL	Nombre de la categoría

materiales — *Catálogo de materiales de construcción*

Campo	Tipo de dato	Restricciones	Descripción
id_material	INTEGER	PK, autoincremental	Identificador único del material
nombre	VARCHAR(100)	NOT NULL	Nombre del material
id_categoria_fk	INTEGER	FK → categorias_material.id_categoria, NOT NULL	Categoría del material
stock_minimo	INTEGER	DEFAULT 0	Umbral mínimo antes de generar alerta
unidad_medida	VARCHAR(20)	NOT NULL	Unidad de medida (kg, m3, unid, etc.)

inventario_global — *Stock disponible a nivel general (bodega central)*

Campo	Tipo de dato	Restricciones	Descripción
id_inventario	INTEGER	PK, autoincremental	Identificador único
id_material_fk	INTEGER	FK → materiales.id_material	Material relacionado
stock_actual	INTEGER	DEFAULT 0	Cantidad disponible en bodega central

Campo	Tipo de dato	Restricciones	Descripción
unidad_medida	VARCHAR(20)	NOT NULL	Unidad de medida

inventario_proyecto — Stock de materiales asignado a un proyecto específico

Campo	Tipo de dato	Restricciones	Descripción
id_inv_proy	INTEGER	PK, autoincremental	Identificador único
id_proyecto_fk	INTEGER	FK → proyectos.id_proyecto	Proyecto relacionado
id_material_fk	INTEGER	FK → materiales.id_material	Material relacionado
stock_actual	INTEGER	DEFAULT 0	Cantidad disponible en el frente de obra
unidad_medida	VARCHAR(20)	NULL	Unidad de medida

solicitudes_material — Solicitudes de material realizadas por un proyecto

Campo	Tipo de dato	Restricciones	Descripción
id_solicitud	INTEGER	PK, autoincremental	Identificador único
id_proyecto_fk	INTEGER	FK → proyectos.id_proyecto	Proyecto solicitante
id_material_fk	INTEGER	FK → materiales.id_material	Material solicitado
cantidad_solicitada	INTEGER	NOT NULL	Cantidad requerida
estado_solicitud	VARCHAR(50)	DEFAULT 'pendiente'	pendiente aprobado rechazado insuficiente
fecha_solicitud	DATETIME	DEFAULT now()	Fecha de la solicitud

notificaciones — Notificaciones y alertas del sistema

Campo	Tipo de dato	Restricciones	Descripción
id_notificacion	INTEGER	PK, autoincremental	Identificador único
titulo	VARCHAR(255)	NOT NULL	Título de la notificación
mensaje	TEXT	NOT NULL	Contenido del mensaje
tipo	VARCHAR(50)	NULL	alarma_stock aviso
leida	BOOLEAN	DEFAULT FALSE	Indica si ya fue leída
fecha_creacion	DATETIME	DEFAULT now()	Fecha de creación
id_material_fk	INTEGER	FK → materiales.id_material, NULL	Material relacionado (si aplica)
stock_actual	INTEGER	NULL	Stock al momento de la alerta
unidad_medida	VARCHAR(20)	NOT NULL, DEFAULT 'unid'	Unidad de medida

archivos_proyecto — Archivos adjuntos (planos, documentos, evidencias) por proyecto o tarea

Campo	Tipo de dato	Restricciones	Descripción
id_archivo	INTEGER	PK, autoincremental	Identificador único
id_proyecto_fk	INTEGER	FK → proyectos.id_proyecto, NULL	Proyecto asociado
id_tarea_fk	INTEGER	FK → tareas.id_tarea, NULL	Tarea asociada
nombre_archivo	VARCHAR(255)	NOT NULL	Nombre del archivo almacenado
tipo_mime	VARCHAR(50)	NOT NULL	Tipo MIME del archivo
ruta_url	VARCHAR(500)	NOT NULL	Ruta o URL de almacenamiento
tamano_bytes	INTEGER	NOT NULL	Tamaño del archivo en bytes
fecha_subida	DATETIME	DEFAULT now()	Fecha de subida

Scripts de instalación

El sistema se despliega completamente mediante Docker. A continuación se listan los archivos de configuración y scripts utilizados.

docker-compose.yml

```
services:
  backend:
    build:
      context: .
      dockerfile: Dockerfile.backend
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app/backend
      - ./database:/app/database
    env_file:
      - ./backend/.env

  frontend:
    build:
      context: .
      dockerfile: Dockerfile.frontend
    ports:
      - "8080:80"
    volumes:
      - ./frontend:/usr/share/nginx/html
    depends_on:
      - backend
```

Dockerfile.backend

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
WORKDIR /app/backend
EXPOSE 8000
CMD sh -c "python seed.py && uvicorn main:app --host 0.0.0.0 --port 8000 --reload"
```

Dockerfile.frontend

```
FROM nginx:alpine
COPY frontend/ /usr/share/nginx/html/
EXPOSE 80
```

requirements.txt (dependencias del backend)

```
fastapi
sqlalchemy
uvicorn
pydantic-settings
pydantic[email]
python-jose[cryptography]
passlib[bcrypt]
bcrypt==4.0.1
python-multipart
python-dotenv
```

7.5 start.sh — inicia el sistema

```
#!/bin/bash
set -e
cd "$(dirname "$0")"

if [ ! -f backend/.env ]; then
    echo "Creando backend/.env con valores por defecto..."
    cat > backend/.env << 'EOF'
    SECRET_KEY=super_secret_key_constructora_gg_2026
    ALGORITHM=HS256
    ACCESS_TOKEN_EXPIRE_MINUTES=60
    EOF
fi

docker compose up -d --build
echo "Backend: http://localhost:8000"
echo "Frontend: http://localhost:8080"
```

stop.sh — detiene el sistema

```
#!/bin/bash
cd "$(dirname "$0")"
docker compose down
```

reset_project.sh — reinicia el proyecto a su estado inicial

```
#!/bin/bash
# Borra la base de datos, limpia archivos subidos y re-ejecuta el seed.

DB_FILE="database/constructora_gg.db"
if [ -f "$DB_FILE" ]; then rm "$DB_FILE"; fi

UPLOADS_DIR="uploads"
if [ -d "$UPLOADS_DIR" ]; then rm -rf "$UPLOADS_DIR"/*; fi

python3 backend/seed.py
```

Pasos de instalación

- 1. Clonar el repositorio: `git clone <url-del-repositorio>`
- 2. Ejecutar: `./start.sh`
- 3. Esperar a que se construyan las imágenes Docker (solo la primera vez o tras cambios en dependencias)
- 4. Abrir el navegador en `http://localhost:8080`
- 5. Para detener el sistema: `./stop.sh`
- 6. Para reiniciar el sistema a su estado inicial: `./reset_project.sh`

Diagrama de componentes

El diagrama de componentes representa la arquitectura de contenedores del sistema (definida en docker-compose.yml) y la forma en que se comunican el frontend, el backend y la base de datos.

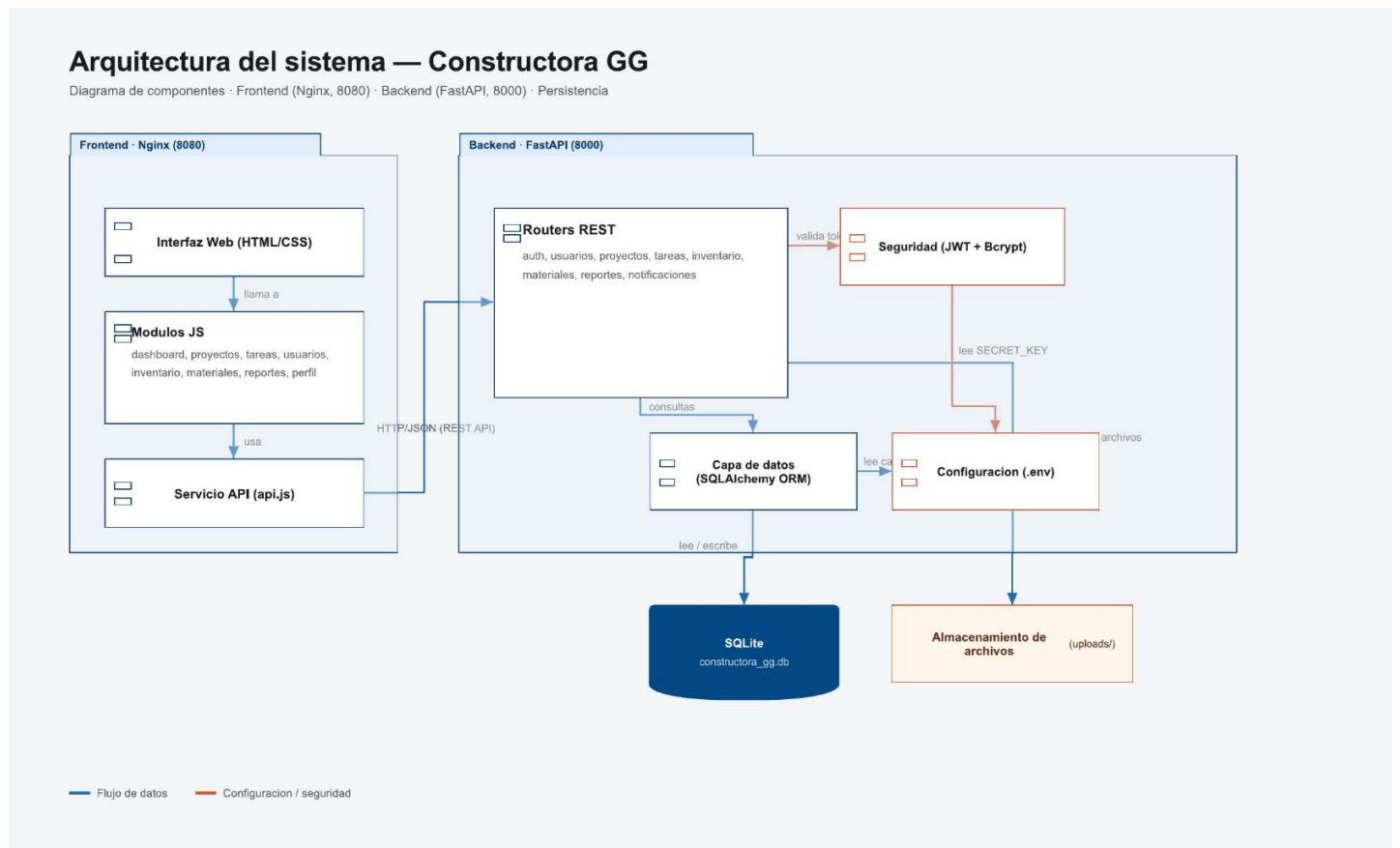


Diagrama de componentes — Sistema Constructora GG